```
Start coding or generate with AI.
```

Module 2 Lab 2: Metrics and Error Costs

Step 1: Manual Metric Calculations

Scenario Selected: S1 — Hospital Sepsis Screening

The purpose of this step is to manually calculate accuracy, precision, recall, and F1 score utilizing the confusion matrix counts provided in the dataset.

Confusion Matrix Counts:

- True Positives (TP): 40
- False Positives (FP): 8
- False Negatives (FN): 2
- True Negatives (TN): 950

Initial Prediction

Before performing the calculations, I predict that accuracy will be approximately 50 percentage points higher than recall because the scenario contains a large number of true negatives.

Manual Calculations

1. Accuracy

Accuracy = (TP + TN) / (TP + FP + FN + TN)

= (40 + 950) / (40 + 8 + 2 + 950)

= 990 / 1000

= 0.99 (99%)

2. Precision

Precision = TP / (TP + FP)

= 40 / (40 + 8)

= 40 / 48

= 0.8333 (83.33%)

3. Recall

Recall = TP / (TP + FN)

= 40 / (40 + 2)

= 40 / 42

= 0.9524 (95.24%)

4. F1 Score

F1 = 2 × (Precision × Recall) / (Precision + Recall)

= 2 × (0.8333 × 0.9524) / (0.8333 + 0.9524)

= 0.8889 (88.89%)

Comparison with Initial Prediction

The actual accuracy is 99%, while recall is approximately 95.24%. Therefore, accuracy is approximately 3.76 percentage points higher than recall.

The prediction was correct about the direction, but the estimated difference of 50 percentage points was much larger than the actual difference.

```
# Step 1: Verify the manually calculated metrics

TP = 40
FP = 8
FN = 2
TN = 950

accuracy = (TP + TN) / (TP + FP + FN + TN)
precision = TP / (TP + FP)
recall = TP / (TP + FN)
f1_score = 2 * (precision * recall) / (precision + recall)

print("Scenario: S1 – Hospital Sepsis Screening")
print(f"Accuracy:  {accuracy:.4f} ({accuracy * 100:.2f}%)")
print(f"Precision: {precision:.4f} ({precision * 100:.2f}%)")
print(f"Recall:    {recall:.4f} ({recall * 100:.2f}%)")
print(f"F1 Score:  {f1_score:.4f} ({f1_score * 100:.2f}%)")
```

```
Scenario: S1 – Hospital Sepsis Screening
Accuracy:  0.9900 (99.00%)
Precision: 0.8333 (83.33%)
Recall:    0.9524 (95.24%)
F1 Score:  0.8889 (88.89%)
```

Step 2: Calculate Metrics Utilizing Python

In this step, I will utilize Python and the pandas library to calculate accuracy, precision, recall, and F1 score for all four scenarios in the dataset. I will then compare my calculated results with the precomputed metric values provided in the CSV file.

```
import pandas as pd

# Load the dataset
df = pd.read_csv("m2_metric_scenarios.csv")

# Display the dataset
display(df)
```

	scenario_id	scenario	true_positive	false_positive	false_negative	tru
0	S1	Hospital sepsis screening	40	8	2	
1	S2	Enterprise spam filter	1200	95	310	
2	S3	Warehouse part defect scan	310	44	26	
3	S4	Loan pre-approval triage	520	130	95	

```
# Calculate accuracy
df["calculated_accuracy"] = (
    (df["true_positive"] + df["true_negative"]) /
    (df["true_positive"] + df["false_positive"] +
     df["false_negative"] + df["true_negative"])
)

# Calculate precision
df["calculated_precision"] = (
    df["true_positive"] /
    (df["true_positive"] + df["false_positive"])
)

# Calculate recall
df["calculated_recall"] = (
    df["true_positive"] /
    (df["true_positive"] + df["false_negative"])
)

# Calculate F1 score
```

```
df["calculated_f1"] = (
    2 * df["calculated_precision"] * df["calculated_recall"] /
    (df["calculated_precision"] + df["calculated_recall"])
)

# Display calculated results
display(df)
```

	scenario_id	scenario	true_positive	false_positive	false_negative	tru
0	S1	Hospital sepsis screening	40	8	2	
1	S2	Enterprise spam filter	1200	95	310	
2	S3	Warehouse part defect scan	310	44	26	
3	S4	Loan pre-approval triage	520	130	95	

```
# Display calculated metrics for all four scenarios

metrics_comparison = df[
    [
        "scenario_id",
        "accuracy",
        "calculated_accuracy",
        "precision",
        "calculated_precision",
        "recall",
        "calculated_recall",
        "f1",
        "calculated_f1"
    ]
]

display(metrics_comparison.round(4))
```

	scenario_id	accuracy	calculated_accuracy	precision	calculated_precisio
0	S1	0.9900	0.9900	0.8333	0.833
1	S2	0.9918	0.9918	0.9266	0.926
2	S3	0.9930	0.9930	0.8757	0.875

3	S4	0.9536	0.9536	0.8000	0.800	

Comparison of Calculated and Precomputed Metrics

The calculated accuracy, precision, recall, and F1 scores for all four scenarios matched the precomputed metric values in the CSV file to four decimal places.

This confirms that the Python calculations were performed correctly utilizing the true positive, false positive, false negative, and true negative counts.

The results demonstrate that the four evaluation metrics can be calculated programmatically and verified against existing values.

Initial Prediction

I predict that S4 — Loan Pre-Approval Triage will have the highest total error cost because it has a relatively high number of false positives (130) and false negatives (95). I expect these errors to create a significant financial impact.

```
# Calculate the cost of false positives and false negatives

df["false_positive_cost"] = (
    df["false_positive"] * df["dollar_cost_per_false_positive"]
)

df["false_negative_cost"] = (
    df["false_negative"] * df["dollar_cost_per_false_negative"]
)

df["total_error_cost"] = (
    df["false_positive_cost"] + df["false_negative_cost"]
)

# Rank scenarios from lowest to highest total error cost
cost_ranking = df.sort_values(
    by="total_error_cost",
    ascending=True
)

display(
    cost_ranking[
        [
            "scenario_id",
            "scenario",
            "false_positive_cost",
            "false_negative_cost",
```

```
            "total_error_cost",
            "accuracy"
        ]
    ]
)
```

	scenario_id	scenario	false_positive_cost	false_negative_cost	total_er
1	S2	Enterprise spam filter	3800	3720	
2	S3	Warehouse part defect scan	3740	23400	
		Loan pre-			

Step 3: Error Cost Results and Ranking

The Python calculations produced the following ranking from lowest to highest total error cost:

1. S2 — Enterprise Spam Filter: $7,520
2. S3 — Warehouse Part Defect Scan: $27,140
3. S4 — Loan Pre-Approval Triage: $343,000
4. S1 — Hospital Sepsis Screening: $500,008

My initial prediction was that S4 would have the highest total error cost. However, the actual results showed that S1 had the highest cost because every false negative costs $250,000. This demonstrates that the financial impact of an error depends not only on the number of errors but also on the cost assigned to each error.

Comparison of Cost Ranking and Accuracy Ranking

The accuracy ranking differs from the total error cost ranking. S3 contains the highest accuracy at 99.30%, while S1 has the highest total error cost at 500,008. S2 has the lowest total error cost at $7,520, despite not having the highest accuracy.

This demonstrates that accuracy alone cannot determine the financial impact or operational suitability of a classification model.

Step 4: Selecting the Appropriate Evaluation Metrics

Different scenarios need different evaluation metrics because false positives and false negatives have different financial and operational consequences.

S1 — Hospital Sepsis Screening

S1 — Hospital Sepsis Screening

Recommended metric: Recall

False negatives are costly in this scenario, costing 250,000 each. Recall is necessary because it measures how effectively the model identifies actual positive cases and minimizes missed sepsis cases.

S2 — Enterprise Spam Filter

Recommended metric: Precision

False positives cost 40 each, compared with 12 for false negatives. Precision is important because it measures how many messages identified as spam are actually spam, assisting to reduce legitimate emails being incorrectly classified as spam.

S3 — Warehouse Part Defect Scan

Recommended metric: Recall

False negatives cost 900 each, compared with 85 for false positives. Recall is important because missing defective products could permit faulty items to pass through quality control.

S4 — Loan Pre-Approval Triage

Recommended metric: Recall

False negatives cost 3,200 each, compared with 300 for false positives. Recall is necessary because the model should identify as many potentially eligible applicants as possible and avoid incorrect rejection.

Step 5: Find the Cost-Minimizing Threshold for S4

This step evaluates different classification threshold settings for S4 — Loan Pre-Approval Triage.

The starting configuration has 130 false positives and 95 false negatives. Each false positive costs 300, while each false negative costs 3,200.

The instructor's model assumes that each threshold adjustment increases false positives by 4 and decreases false negatives by 1.

The objective is to test at least five threshold settings, calculate their total error costs, and identify the setting that minimizes the financial cost.

Total Error Cost = (False Positives × 300) + (False Negatives × 3,200)

```
# Threshold optimization for S4

fp, fn = 130, 95
cost_fp, cost_fn = 300, 3200

for step in range(0, 6):
    f_p = fp + 4 * step
    f_n = fn - step

    total = (f_p * cost_fp) + (f_n * cost_fn)

    print(step, f_p, f_n, total)
```

```
0 130 95 343000
1 134 94 341000
2 138 93 339000
3 142 92 337000
4 146 91 335000
5 150 90 333000
```

Threshold Optimization Results

The six tested threshold settings produced different total error costs.

The lowest cost among the tested settings occurred at setting 5, with 150 false positives and 90 false negatives.

The total error cost was:

(150 × 300) + (90 × 3,200)

= 45,000 + 288,000

= 333,000

The original configuration had a total error cost of $343,000.

Cost Savings:

343,000 − 333,000 = 10,000

Therefore, setting 5 reduced the total error cost by $10,000 compared with the original configuration.

GitHub Portfolio Repository

Repository URL: https://github.com/seneviratnemiyuru01-source/MO2L2-Metrics-and-Error-Costs